
ALGORITMOS E ESTRUTURA DE DADOS

Teoria e Implementações

J. P. S. DE SOUSA

Sumário

I	Estruturas de Dados Elementares	1
1	Estruturas Sequenciais	2
1.1	Vetores	3
1.1.1	Vetores Não Ordenados	3
1.1.2	Vetores Ordenados	4
1.2	Listas Encadeadas	6
1.2.1	Lista Simplesmente Encadeada	7
1.2.2	Lista Duplamente Encadeada	8
1.2.3	Lista Circular Duplamente Encadeada	9

PARTE I

ESTRUTURAS DE DADOS ELEMENTARES

Estruturas Sequenciais

Na matemática, conjuntos são coleções imutáveis de objetos distintos. Na ciência da computação, conferimos a nomenclatura de **conjuntos** a qualquer coleção finita abstrata de objetos. A imutabilidade, nesse caso, não é um requisito; conjuntos computacionais podem crescer ou diminuir de tamanho ao longo das operações de um algoritmo que o manipula. Tais conjuntos que podem ter seu tamanho alterado chamamos de **dinâmicos**. Os conjuntos dinâmicos que também permitem o teste de pertinência de um membro dado são chamados de **dicionários**.

Os elementos de um conjunto de dados são, geralmente, um registro, cujos campos podem ser de tipos elementares, ponteiros, ou compostos de outros registros. Os elementos num conjunto não necessariamente precisam ser todos distintos entre si, a exemplo da coleção de idades de um grupo de pessoas; nada impede que haja duas com a mesma idade. Em alguns casos, os membros de um conjunto possuem uma **chave** que os identifica, e, caso as chaves sejam todas distintas, o conjunto pode ser considerado como uma coleção de chaves.

As operações de um conjunto podem ser de dois tipos: consulta ou modificação. A primeira apenas resgata um valor, membro ou estatística do conjunto. A segunda altera o estado do conjunto, como sua ordenação, inserindo, removendo ou modificando membros, entre outras. Em geral, quase todo conjunto aceita as seguintes operações:

Insere(S, x) Operação que toma um conjunto S e insere o membro apontado por x . Presume-se que tanto S quanto x já foram inicializados. Dependendo da aplicação, a operação pode falhar caso um membro x com a mesma chave já exista em S .

Remove(S, x) Remove do conjunto S o membro apontado por x .

Busca(S, k) Retorna um ponteiro para um membro x em S cuja chave seja igual a k , ou **NIL** caso nenhum elemento contenha tal chave.

Outras operações básicas que acompanham conjuntos dinâmicos são

Máximo(S) Retorna o membro x de S cujo valor da chave é máximo no conjunto

Mínimo(S) Retorna o membro x de S cujo valor da chave é mínimo no conjunto

Predecessor(S, x) Num conjunto ordenado, retorna o membro $w \in S$ que precede x , ou **NIL** quando x é o elemento mínimo.

Sucessor(S, x) Num conjunto ordenado, retorna o membro $y \in S$ que sucede x , ou **NIL** quando x é o elemento máximo.

1.1 Vetores

Um vetor é um conjunto de objetos de mesmo tipo¹ que são armazenados contiguamente na memória. Seus elementos podem ser acessados diretamente pelos seus índices. Se o nome do vetor é S , então acessamos seu i -ésimo membro como $S[i]$. O nome do vetor é um ponteiro para o seu primeiro elemento, e, ao somá-lo com um valor inteiro não negativo i , obtemos um ponteiro para o elemento de índice i do vetor, portanto $S[i] = *(S + i)$.

Na sua formulação mais simples, vetores possuem dois atributos principais: tamanho – o número máximo de elementos que ele suporta – e o número efetivo de membros. Chamamos de estáticos os vetores que possuem tamanho previamente determinado e fixo, enquanto são denominados dinâmicos aqueles cujo tamanho varia conforme a necessidade da aplicação. Além disso, os vetores podem ser ordenados ou não.

1.1.1 Vetores Não Ordenados

A busca em vetores não ordenados é realizada percorrendo o vetor sequencialmente até achar o elemento com a chave desejada. No pior caso, o elemento ou está na última posição ou não existe no vetor. Com isso, a complexidade de uma busca sequencial no vetor com n elementos é $O(n)$.

Função Busca-Vetor(S, k)	
1	$i \leftarrow 0$;
2	enquanto $i < S.\text{num_elem}$ e $S[i].\text{chave} \neq k$ faça
3	$i \leftarrow i + 1$;
4	fim
5	se $i = S.\text{num_elem}$ então
6	retorna NIL ;
7	fim
8	retorna $\&S[i]$;

A inserção em vetores não ordenados pode ser feita apenas inserindo após o último elemento, tendo, pois, custo de tempo constante caso o vetor não esteja cheio. Senão, será necessário realocá-lo para uma nova região da memória, movendo todos os seus membros. Com isso, o custo de pior caso de uma inserção é $O(n)$.

¹Isso porque dois dados do mesmo tipo possuem o mesmo número de bytes na memória.

Função Insere-Vetor(S, x)

```
1 se  $S.len = S.num\_elem$  então
2   |  $S \leftarrow \text{Realoca-Vetor}(S)$ ;
3 fim
4 copia( $S + S.num\_elem, x$ );
5  $S.num\_elem \leftarrow S.num\_elem + 1$ ;
```

Se toda vez que realoquemos o vetor dobrarmos o seu tamanho, o custo amortizado da operação será reduzido. Cada realocação de n elementos é precedida por n inserções com custo constante, enquanto que a próxima realoca $2n$ elementos após n novas inserções. Desse modo, a quantidade de elementos realocados na k -ésima realocação é

$$\sum_{i=0}^{k-1} 2^i \cdot n = n(2^k - 1)$$

Portanto, o custo amortizado da realocação em relação às inserções é

Função Reloca-Vetor(S)

```
1  $T \leftarrow \text{aloca}(S.len * 2)$ ;
2 copia( $T, S$ );
3  $T.num\_elem \leftarrow S.num\_elem$ ;
4  $T.len \leftarrow S.len$ ;
5 retorna  $T$ ;
```

A operação de remoção, por sua vez pode ser realizada simplesmente copiando o último elemento para a posição do elemento que está sendo removido e decrementando o atributo de contagem de membros. Caso uma nova inserção seja feita, o elemento que antes ocupava a última posição será sobrescrito. A complexidade é $O(1)$.

Função Remove-Vetor(S, x)

```
//  $x$  deve apontar para um elemento do vetor:
 $S \leq x < S + S.num\_elem$ 
1 se  $S.num\_elem = 0$  então
2   | erro “underflow”;
3 fim
4  $S.num\_elem \leftarrow S.num\_elem - 1$ ;
5 ultimo  $\leftarrow S + S.num\_elem$ ;
6 copia( $x, ultimo$ );
```

1.1.2 Vetores Ordenados

Em vetores ordenados, a busca pode ser feita em tempo sublinear $O(\log n)$ se usarmos o algoritmo de busca binária, que será apresentado num capítulo mais adiante. Uma abordagem possível para a inserção é inserir o novo membro ao final do vetor, e o movimentar na direção decrescente enquanto sua chave for menor que a do membro na posição anterior. Para remoção, o procedimento consiste em deslocar todos os membros

à frente daquele que foi removido para uma posição atrás. Ambas as operações passam a ter, pois, complexidade $O(n)$, já que necessitam mover todos os membros do vetor no pior caso.

Função Insere-Vetor-Ordenado(S, x)

```
1 se  $S.len = S.num\_elem$  então
2   | erro “overflow”;
3 fim
4 copia( $S + S.num\_elem, x$ );
5  $i \leftarrow S.num\_elem$ ;
6 enquanto  $i \geq 1$  e  $S[i].chave < S[i - 1].chave$  faça
7   |  $temp \leftarrow S[i - 1]$ ;
8   |  $S[i - 1] \leftarrow S[i]$ ;
9   |  $S[i] \leftarrow temp$ ;
10  |  $i \leftarrow i - 1$ ;
11 fim
12  $S.num\_elem \leftarrow S.num\_elem + 1$ ;
```

Função Remove-Vetor-Ordenado(S, x)

```
1 se  $S.num\_elem = 0$  então
2   | erro “underflow”;
3 fim
4 para  $i \leftarrow x - S$  até  $S.num\_elem - 2$  faça
5   |  $S[i] \leftarrow S[i + 1]$ ;
6 fim
7  $S.num\_elem \leftarrow S.num\_elem - 1$ ;
```

Operações para obter o máximo e o mínimo, basta retornar o ponteiro para o último e o primeiro elemento no vetor respectivamente. Para obter o predecessor de um elemento, devemos apenas subtrair 1 ao ponteiro dado como argumento caso esse não seja o último. Para o sucessor, devemos somar 1 a esse ponteiro, caso ele não aponte para o último. Todas essas opções são realizadas em tempo constante.

```
1 função Maximo( $S$ )
2   se  $S.num\_elem = 0$  então
3     retorna NIL;
4   fim
5   retorna  $S + S.num\_elem - 1$ ;
6 fim
7 função Minimo( $S$ )
8   se  $S.num\_elem = 0$  então
9     retorna NIL;
10  fim
11  retorna  $S$ ;
12 fim
13 função Predecessor( $S, x$ )
14   se  $x = Minimo(S)$  então
15     retorna NIL;
16   fim
17   retorna  $x - 1$ ;
18 fim
19 função Sucessor( $S, x$ )
20   se  $x = Maximo(S)$  então
21     retorna NIL;
22   fim
23   retorna  $x + 1$ ;
24 fim
```

1.2 Listas Encadeadas

Lista encadeada é uma implementação de conjunto dinâmico cujos membros, denominados nós, nas suas formas mais simples, detêm um campo de chave e um ponteiro *proximo* que aponta para o próximo membro no conjunto. Numa implementação típica, a lista possui dois ponteiros especiais: *head* e *tail*. O *head* aponta para o primeiro elemento, enquanto o segundo para o último. No entanto, para simplificar condições de contorno nas operações de inserção e remoção, principalmente no início da lista, adiciona-se um nó chamado de *sentinela*, que não possui chave válida, é apontado pelo *head* e cujo *proximo* aponta para o primeiro membro efetivo da lista.

As listas encadeadas possuem diversas variações: listas simplesmente encadeadas, duplamente encadeadas e circulares. Também podemos contar com variações que incluem elementos de mais de um tipo. De forma geral, assim como em vetores não ordenados, é possível realizar inserções e remoções de um dado membro em tempo constante caso a ordenação não seja requisitada, e buscas em tempo linear.

As listas encadeadas possuem a vantagem de serem mais flexíveis e permitirem inserções sem necessariamente conhecer o total de elementos a serem comportados, diferentemente dos vetores, que precisam ser realocados à medida que o número de

membros cresce. No entanto, os vetores fazem melhor aproveitamento da cache devido ao princípio de localidade espacial. No caso de existência de ordenação, a vantagem das listas encadeadas diminui mais, pois uma busca num vetor ordenado pode ser executada em tempo logarítmico, enquanto nas listas continua sendo realizado em tempo linear.

1.2.1 *Lista Simplesmente Encadeada*

Consiste na implementação mais simples para uma lista encadeada. A lista mantém três atributos: `head`, `tail` e o número de elementos. Uma busca nessa lista consiste numa pesquisa sequencial: percorremos cada elemento, seguindo os ponteiros de `proximo` até obter a chave desejada. É importante frisar que o que é retornado não é um ponteiro para o membro cuja chave é aquela sendo pesquisada, mas sim um ponteiro para o membro anterior. Isso porque um nó não conhece aquele que o antecede, apenas o que lhe sucede.

Função Busca-Lista(<i>L</i> , <i>k</i>)
<pre>1 <i>w</i> ← <i>L</i>.head; 2 enquanto <i>w</i>.proximo ≠ NIL e <i>w</i>.proximo.chave ≠ <i>k</i> faça 3 <i>w</i> ← <i>w</i>.proximo; 4 fim 5 se <i>w</i>.proximo = NIL então 6 retorna NIL; 7 fim 8 retorna <i>w</i>;</pre>

A inserção é feita tomando-se como argumento o nó `prev` que antecederá o novo membro do conjunto. Neste momento, é feito o ajuste dos ponteiros de `proximo`, a fim de que o `proximo` de `prev` aponte para o novo membro, e o `proximo` desse último aponte para o antigo sucessor de `prev`. Se `prev` era o `tail` da lista, então o novo nó será o novo `tail`, o que pode ser feito verificando-se se o `proximo` do novo é `NIL`.

Função Insere-Lista(<i>L</i> , <i>prev</i> , <i>x</i>)
<pre>1 <i>x</i>.proximo ← <i>prev</i>.proximo; 2 <i>prev</i>.proximo ← <i>x</i>; 3 se <i>x</i>.proximo = NIL então 4 <i>L</i>.tail ← <i>x</i>; 5 fim 6 <i>L</i>.num_elem ← <i>L</i>.num_elem + 1;</pre>

A remoção toma como argumento o nó `prev` que precede aquele que será removido. Nessa operação, o `proximo` de `prev` irá apontar para o `proximo` do nó retirado. Se o removido era o `tail`, então `prev` passa a ocupar essa função.

Função Remove-Lista(L , prev)	
1	se prev.proximo = NIL então
2	retorna;
3	fim
4	$x \leftarrow \text{prev.proximo};$
5	$\text{prev.proximo} \leftarrow x.\text{proximo};$
6	se prev.proximo = NIL então
7	$L.\text{tail} \leftarrow \text{prev};$
8	fim
9	$L.\text{num_elem} \leftarrow L.\text{num_elem} - 1;$

1.2.2 Lista Duplamente Encadeada

Essa variante funciona de maneira análoga à encadeada simples. O adicional é que os membros agora possuem um ponteiro de anterior, que aponta para o nó antecessor. Nesse caso, a busca pode retornar um ponteiro para o próprio elemento que possui a chave pesquisada.

Função Busca-Lista-D(L , k)	
1	$w \leftarrow L.\text{head.proximo};$
2	enquanto $w \neq \text{NIL}$ e $w.\text{chave} \neq k$ faça
3	$w \leftarrow w.\text{proximo};$
4	fim
5	se $w = \text{NIL}$ então
6	retorna NIL;
7	fim
8	retorna w ;

Função Insere-Lista-D(L , prev, x)	
1	$x.\text{proximo} \leftarrow \text{prev.proximo};$
2	$\text{prev.proximo} \leftarrow x;$
3	$x.\text{anterior} \leftarrow \text{prev};$
4	se $x.\text{proximo} = \text{NIL}$ então
5	$L.\text{tail} \leftarrow x;$
6	senão
7	$x.\text{proximo.anterior} \leftarrow x;$
8	fim
9	$L.\text{num_elem} \leftarrow L.\text{num_elem} + 1;$

Função Remove-Lista-D(L, x)	
1	se $x = L.head$ então
2	retorna;
3	fim
4	$prev \leftarrow x.anterior$;
5	$prev.proximo \leftarrow x.proximo$;
6	se $prev.proximo = NIL$ então
7	$L.tail \leftarrow prev$;
8	senão
9	$x.proximo.anterior \leftarrow prev$;
10	fim
11	$L.num_elem \leftarrow L.num_elem - 1$;

1.2.3 Lista Circular Duplamente Encadeada

A lista encadeada circular é aquela em que o último nó aponta para o primeiro em vez de **NIL**, ou seja, o **head** da lista equivale a $L.tail.proximo$. Ao adicionar o ponteiro **anterior** a cada membro, obtemos, então, a lista circular duplamente encadeada, que combina todos os elementos vistos até então sobre listas. As operações de busca, inserção e remoção são análogas àquelas vistas para listas duplamente encadeadas.

Iremos expor a implementação dessa variação usando o nó sentinela. Com a lista vazia, os ponteiros **proximo** e **anterior** da sentinela apontam para ela mesma. À medida que as inserções ocorrem, esses ponteiros são ajustados para refletir a estrutura circular.

A busca continua retornando um ponteiro para o nó que contém a chave desejada. O que muda é a condição de parada quando a chave não existe. Em vez de encerrar quando atingir **NIL**, o critério de parada passa a ser visitar a sentinela.

Função Busca-Lista-DC(L, k)	
1	$w \leftarrow L.head.proximo$;
2	enquanto $w \neq L.head$ e $w.chave \neq k$ faça
3	$w \leftarrow w.proximo$;
4	fim
5	se $w = L.head$ então
6	retorna NIL ;
7	fim
8	retorna w ;

A estrutura circular elimina a condição de contorno de quando inserimos ou eliminamos no final da lista. Desse modo, as inserções e remoções funcionam da mesma forma independentemente de onde as realizamos na lista. A única operação que deve ser coibida é a remoção da própria sentinela.

Função Insere-Lista-DC(L, prev, x)

```
1  $x.\text{proximo} \leftarrow \text{prev}.\text{proximo}$  ;  
2  $\text{prev}.\text{proximo} \leftarrow x$ ;  
3  $x.\text{anterior} \leftarrow \text{prev}$ ;  
4  $x.\text{proximo}.\text{anterior} \leftarrow x$ ;  
5  $L.\text{num\_elem} \leftarrow L.\text{num\_elem} + 1$ ;
```

Função Remove-Lista-DC(L, x)

```
1 se  $x = L.\text{head}$  então  
2   | retorna;  
3 fim  
4  $\text{prev} \leftarrow x.\text{anterior}$ ;  
5  $\text{prev}.\text{proximo} \leftarrow x.\text{proximo}$ ;  
6  $x.\text{proximo}.\text{anterior} \leftarrow \text{prev}$ ;  
7  $L.\text{num\_elem} \leftarrow L.\text{num\_elem} - 1$ ;
```
